Universidad del Valle de Guatemala
Facultad de Ingeniería
Programación Orientada a Objetos
Segundo Ciclo, 2026
Catedrático: Erick Marroquin

Laboratorio No. 4

Polimorfismo

Nombre: Raul Escobar
Carné: 26429
Sección # 30
Fecha de entrega: 06/11/2026

Análisis

Este documento define una solución orientada a objetos para administrar equipos, encuentros de fútbol sala y baloncesto, exhibiciones, calendario y tabla de posiciones. Las reglas deportivas se resuelven en subclases; el calendario usa un contrato común para todos los eventos y la recepción de resultados utiliza un contrato independiente.

Alcance: análisis y diseño. Incluye tablas editables y diagramas UML a color insertados en el documento. No contiene implementación Java ni evidencia o afirmaciones de pruebas ejecutadas. Los ejemplos son resultados esperados deducidos del enunciado.

Fuente y criterio de interpretación

Fuente: Lab_4_Polimorfismo_Enunciado_4_ArenaGT_Lite.pdf, seis páginas. Las páginas 1 y 2 definen el caso y las pautas; la página 3 contiene los seis puntos de análisis y los requisitos UML; la página 4 aporta restricciones de operación; la página 6 contiene la rúbrica. Las decisiones adicionales se identifican expresamente como decisiones de diseño.

Cobertura de los seis puntos del análisis

Punto solicitado	*Sección que lo desarrolla*
1 Requisitos y validez	1 y 2 · Datos, operaciones, validaciones y atomicidad.
2 Reglas y abstracciones	3 y 5 · Reglas por deporte y reparto de responsabilidades.
3 Estados y cambios	4 · Estados derivados, operaciones y transiciones rechazadas.
4 Clases y miembros	5 a 10 · Responsabilidades, atributos, métodos y constructores tipados.
5 Dos interfaces	11 · Separación de contratos y consecuencias de fusionarlos.
6 Extensibilidad	11 · Nuevo deporte y nuevos eventos sin alterar los algoritmos de consulta.

Convenciones de diseño

“Puntos de tabla” es la acumulación de cada equipo; “marcador” son los goles o puntos de un encuentro. + significa público, - privado, # protegido y ~ acceso de paquete. Los tipos y firmas son especificaciones del diseño, no código implementado. Las clases del dominio pertenecen al mismo paquete; Main pertenece al paquete de presentación.

1. Requisitos funcionales

ID	*Información u operación*	*Condiciones y efecto requerido*
RF01	Registrar equipos	Conservar código único, nombre y empresa. Puntos de tabla iniciales en 0. Rechazar código vacío o repetido.
RF02	Crear encuentros	Elegir deporte y dos equipos registrados y distintos. Conservar sus referencias. Iniciar sin programación ni marcador.
RF03	Crear exhibiciones	Conservar título. Iniciar sin programación. No admitir equipos, marcador ni resultado.
RF04	Programar eventos	Asignar fecha válida, hora entera entre 0 y 23 y sede. Verificar ocupación de sede y, para los participantes, ocupación de equipos.
RF05	Reprogramar eventos	Sustituir fecha, hora y sede únicamente si no hay resultado. Excluir al mismo evento al buscar conflictos. Conservar la programación anterior si hay rechazo.
RF06	Registrar resultados	Solo encuentro programado; dos enteros no negativos; aplicar regla deportiva. Guardar marcador, sumar puntos a ambos equipos y finalizar una sola vez.
RF07	Consultar posiciones	Todos los equipos, incluidos los de 0 puntos. Ordenar por puntos de tabla descendentes y nombre ascendente.
RF08	Consultar calendario	Solo eventos programados, incluidos encuentros finalizados; mezclar encuentros y exhibiciones. Ordenar por fecha, hora y sede.
RF09	Recorrer calendario	Copa implementa Iterable<Programable>; entregar un iterador sobre una copia ordenada e inmodificable. Ocultar el almacenamiento interno.
RF10	Interacción futura	Menú persistente para altas, creación, programación, resultados y consultas. Salir a petición del usuario. Informar errores y permitir reintento.
RF11	Datos de inicio futuros	Al menos 4 equipos de 2 empresas, 4 encuentros con ambos deportes, 2 con resultado y 1 exhibición programada. Incluir empate de fútbol y desempate por nombre.
RF12	Conservación	Solo memoria durante la ejecución. No se requieren archivos, base de datos ni interfaz gráfica.

RF10 y RF11 se especifican para mantener coherencia con la solución futura; este entregable no implementa el menú ni carga datos en un programa.

2. Validaciones y conservación del estado

Entrada o situación	*Validación y responsable*	*Efecto ante rechazo*
Código de equipo	Copa normaliza con trim y exige valor no vacío y no registrado. Equipo verifica sus textos.	No se agrega el equipo.
Identificador de evento	Copa exige id no vacío y único entre todos sus eventos.	No se agrega el evento ni se consume ningún resultado.
Equipos del encuentro	Copa resuelve ambos códigos; Encuentro exige referencias no nulas y distintas.	No se crea el encuentro.
Número o fecha mal escritos	Main convierte entradas y captura errores de formato. El dominio recibe tipos ya convertidos.	Mensaje claro y reintento; no se invoca la operación del dominio.
Hora o marcador	Programacion exige hora 0–23. Encuentro exige marcadores enteros ≥ 0; subclase valida la regla deportiva.	Se conservan agenda, marcador, estado y puntos.
Fecha, sede o título	Fecha no nula y válida; sede y título no vacíos. Sin restricción de fecha futura porque no la exige el PDF.	No se confirma el alta o la programación.
Sede ocupada	Copa.validarAgenda compara fecha, hora y sede con todos los otros eventos programados.	La agenda anterior permanece intacta.
Equipo ocupado	Copa.validarAgenda comprueba intersección de participantes en esa fecha y hora, sin consultar clases concretas.	La agenda anterior permanece intacta.
Resultado prematuro o repetido	Encuentro exige estado PROGRAMADO antes de validar y calcular puntos.	No cambia el marcador ni se suman puntos.
Reprogramación tras finalizar	Encuentro.programar rechaza si el estado es FINALIZADO.	Fecha, hora, sede y puntos permanecen iguales.

Orden de validación y actualización

Programación: construir una Programacion válida → verificar que el evento pertenece a la copa y permite el cambio → verificar todos los conflictos excluyendo el propio evento → sustituir la única referencia de Agenda. No se retira primero la reserva anterior.

Resultado: validar estado y ambos marcadores → ejecutar validación deportiva → calcular los puntos de ambos equipos → comprobar que ambas sumas caben en int → guardar ambos marcadores y sumar ambos valores. Todo rechazo previsto ocurre antes de la primera escritura. Se presupone una aplicación de consola de un solo hilo; no se diseña concurrencia.

Decisiones adicionales: códigos e identificadores se comparan exactamente después de trim, distinguiendo mayúsculas. Los nombres y las sedes se ordenan con String.compareTo; la sede se normaliza con trim. Nombres y empresas no vacíos, ids de eventos y control de desbordamiento son precisiones de diseño. No se modelan duración ni solapamientos entre horas distintas.

3. Reglas deportivas y comportamiento común

Regla	*Fútbol sala*	*Baloncesto*	*Responsable*
Marcador base	Dos enteros ≥ 0	Dos enteros ≥ 0	Encuentro valida la condición común.
Igualdad	Empate permitido	Empate rechazado	validarDeporte en cada subclase.
Victoria	3 puntos de tabla	2 puntos de tabla	puntosPara en cada subclase.
Empate	1 punto para cada equipo	No se reparte ningún punto	puntosPara / validarDeporte.
Derrota	0 puntos de tabla	1 punto de tabla	puntosPara en cada subclase.

Comportamiento	*Abstracción elegida*	*Motivo*
Fecha, hora, sede y orden	Interfaz Programable y clase Programacion	Contrato compartido por objetos que no pertenecen a la misma jerarquía.
Reserva y conflictos	Agenda y Copa concretas	Agenda concentra la actualización; Copa conoce todos los eventos para detectar conflictos.
Marcador, participantes y ciclo del encuentro	Clase abstracta Encuentro	Reutiliza datos y algoritmo, e impide crear un encuentro sin reglas deportivas.
Validación y puntos de cada deporte	FutbolSala y Baloncesto concretas	Resuelven los dos métodos protegidos abstractos mediante polimorfismo.
Recibir resultado	Interfaz Puntuable	Contrato exclusivo de los encuentros; una exhibición no puede cumplirlo con sentido.
Título y ausencia de participantes	Clase Exhibicion	Solo cumple Programable y devuelve un conjunto vacío de equipos ocupados.

Ejemplos de cálculo esperado

Faro 3–1 Sol en fútbol sala aporta Faro +3 y Sol +0. Sol 78–74 Faro en baloncesto aporta Sol +2 y Faro +1. Acumulación esperada: Faro 4 y Sol 2. Un 2–2 de fútbol aporta +1 a cada equipo; un 74–74 de baloncesto se rechaza sin aportes. Son deducciones de las reglas, no resultados de una ejecución.

4. Estados y transiciones

El estado se deriva de los datos para evitar inconsistencias: en Encuentro, la presencia de marcador indica FINALIZADO; sin marcador, la existencia de programación distingue PROGRAMADO de SIN_PROGRAMAR. En Exhibicion se deriva únicamente de su programación. El marcador siempre está completo o ausente.

Objeto y origen	*Operación y condición*	*Destino*	*Efecto*
Encuentro inexistente	Construcción válida	SIN_PROGRAMAR	Agenda vacía y marcador ausente; equipos fijados.
Encuentro SIN_PROGRAMAR	programar con fecha, hora y sede sin conflictos	PROGRAMADO	Agenda recibe una programación.
Encuentro PROGRAMADO	programar con nuevos datos válidos	PROGRAMADO	Se reemplaza la programación.
Encuentro PROGRAMADO	registrarResultado válido	FINALIZADO	Marcador completo y puntos acumulados una sola vez.
Encuentro SIN_PROGRAMAR	registrarResultado	Sin cambio	Rechazo por no estar programado.
Encuentro FINALIZADO	programar o registrarResultado	FINALIZADO	Rechazo; no se corrige ni duplica resultado.
Exhibición inexistente	Construcción válida	SIN_PROGRAMAR	Título fijo y agenda vacía.
Exhibición SIN_PROGRAMAR	programar sin conflictos de sede	PROGRAMADA	Agenda recibe una programación.
Exhibición PROGRAMADA	programar sin conflictos de sede	PROGRAMADA	Reprogramación permitida.
Cualquier estado	Entrada o conflicto inválido	Mismo estado	No se modifican datos.

No existen operaciones de cancelar, desprogramar, eliminar equipos o corregir resultados: el enunciado no las solicita. Las exhibiciones no tienen transición a FINALIZADO ni operación de resultado. Un encuentro finalizado conserva su programación y sigue apareciendo en el calendario.

Invariantes de identidad y relaciones

Un encuentro referencia exactamente dos equipos diferentes de la misma copa. Cada evento pertenece a una sola copa y tiene una Agenda exclusiva. Una Agenda tiene cero o una Programacion vigente. Los equipos y eventos se registran una sola vez; las colecciones de Copa almacenan referencias, no copias de sus entidades.

5. Responsabilidades de los tipos

Tipo	*Responsabilidad y justificación*
Programable · interfaz	Define identidad, programación, consulta, participantes ocupados y comparación cronológica. Extiende Comparable<Programable>.
Puntuable · interfaz	Define registrarResultado. Permite solicitar resultados sin depender del deporte; no se aplica a exhibiciones.
Encuentro · abstracta	Implementa ambos contratos, conserva equipos y marcador, deriva estado y ejecuta el algoritmo común de resultados.
FutbolSala / Baloncesto	Concretan validación y puntos; no duplican equipos, agenda ni algoritmo de actualización.
Exhibicion · concreta	Conserva título; implementa programación y devuelve cero participantes. No hereda de Encuentro.
Equipo · concreta	Conserva identidad y acumulado. Implementa Comparable<Equipo> para posiciones.
Copa · concreta	Registra entidades, crea eventos, valida conflictos y expone tabla y calendario como copias. Implementa Iterable<Programable>.
Agenda · auxiliar concreta	Centraliza la sustitución de programación y referencia la copa responsable de validar los conflictos.
Programacion · valor inmutable	Agrupa fecha, hora y sede válidas; evita que una agenda cambie parcialmente.
EstadoEncuentro / EstadoExhibicion · enum	Nombran los estados permitidos; no se guardan como atributos redundantes.
Main · entrada	Futuro menú, conversión de entradas, mensajes y carga inicial por operaciones de Copa. No calcula puntos ni ordena.

Protección del estado

Todos los atributos son privados. Las referencias a identidad, equipos, copa y agenda son finales. No hay setters generales de marcador, puntos, estado o programación. Equipo.acumular tiene acceso de paquete y se usa únicamente por el algoritmo de Encuentro después de validar ambas sumas. Los constructores de entidades registradas tienen acceso de paquete y Copa controla su creación.

Las interfaces tienen miembros públicos por contrato. El código de presentación obtiene vistas inmodificables y no accede a Agenda. El dominio usa tipos de interfaces para las consultas; la elección del deporte ocurre únicamente al crear el encuentro.

6. Atributos de equipos y eventos

Equipo

Atributo con tipo y visibilidad	*Propósito e inicialización*
- codigo: String {final}	Identidad única en Copa; inicializado por constructor.
- nombre: String {final}	Nombre visible y segundo criterio de orden; constructor.
- empresa: String {final}	Empresa representada; constructor.
- puntosTabla: int	Acumulado de clasificación; inicializa en 0.

Encuentro

Atributo con tipo y visibilidad	*Propósito e inicialización*
- id: String {final}	Identificador; constructor.
- equipoA: Equipo {final}	Primer participante registrado; constructor.
- equipoB: Equipo {final}	Segundo participante diferente; constructor.
- agenda: Agenda {final}	Se crea en el constructor común; concentra programación.
- marcadorA: Integer	Inicialmente null; entero no negativo al finalizar.
- marcadorB: Integer	Inicialmente null; se confirma junto al marcador A.

Exhibicion

Atributo con tipo y visibilidad	*Propósito e inicialización*
- id: String {final}	Identidad del evento; constructor.
- titulo: String {final}	Descripción no vacía; constructor.
- agenda: Agenda {final}	Agenda exclusiva creada por constructor.

FutbolSala y Baloncesto no declaran atributos adicionales: heredan el estado común de Encuentro. Programable, Puntuable y Main no poseen atributos de instancia. Las interfaces no almacenan estado.

7. Atributos de coordinación y valores

Copa

Atributo con tipo y visibilidad	*Propósito e inicialización*
- equipos: Map<String, Equipo> {final}	Mapa por código; inicialmente vacío. Un único objeto por equipo.
- eventos: Map<String, Programable> {final}	Mapa por id; inicialmente vacío. Incluye encuentros y exhibiciones.
- puntuables: Map<String, Puntuable> {final}	Índice por id hacia los mismos encuentros de eventos; inicialmente vacío. No duplica entidades ni resultados.

Agenda

Atributo con tipo y visibilidad	*Propósito e inicialización*
- copa: Copa {final}	Copa propietaria; constructor.
- actual: Programacion	Reserva vigente; inicialmente null.

Programacion

Atributo con tipo y visibilidad	*Propósito e inicialización*
- fecha: LocalDate {final}	Fecha válida; constructor.
- hora: int {final}	Hora completa 0–23; constructor.
- sede: String {final}	Sede no vacía normalizada con trim; constructor.

Enumeración pública	*Literales y propósito*
EstadoEncuentro	SIN_PROGRAMAR, PROGRAMADO, FINALIZADO. Tipo del estado derivado de Encuentro.
EstadoExhibicion	SIN_PROGRAMAR, PROGRAMADA. Tipo del estado derivado de Exhibicion.

Las enumeraciones no requieren atributos, métodos ni constructores propios en este diseño. Los miembros provistos automáticamente por Java no se duplican en las tablas ni en UML. Integer permite representar un marcador ausente; Optional se usa únicamente en las consultas, no como almacenamiento redundante.

8. Métodos de las interfaces y del equipo

Programable

Firma con tipo y visibilidad	*Propósito y justificación*
+ getId(): String	Identifica el evento para buscarlo sin usar su clase.
+ getProgramacion(): Optional<Programacion>	Consulta inmutable; vacío significa sin programar.
+ getEquiposOcupados(): Set<Equipo>	Conjunto inmodificable; dos en encuentros, vacío en exhibiciones. Permite conflictos sin instanceof.
+ getDescripcion(): String	Descripción para mostrar el calendario mediante polimorfismo.
+ programar(fecha: LocalDate, hora: int, sede: String): void	Programa o reprograma; implementación verifica estado y delega en Agenda.
+ compareTo(otro: Programable): int {default}	Único criterio cronológico. Programados primero por fecha, hora y sede; los no programados quedan al final. Dos no programados comparan 0.

Puntuable

Firma con tipo y visibilidad	*Propósito y justificación*
+ registrarResultado(a: int, b: int): void	Recibe marcador, valida reglas y reparte puntos una sola vez. El primer valor corresponde al equipo A.

Equipo

Firma con tipo y visibilidad	*Propósito y justificación*
+ getCodigo(): String	Consulta identidad sin permitir cambios.
+ getNombre(): String	Consulta nombre.
+ getEmpresa(): String	Consulta empresa.
+ getPuntosTabla(): int	Consulta acumulado.
~ acumular(puntos: int): void	Suma aporte no negativo; uso interno después de comprobar ambas sumas.
+ compareTo(otro: Equipo): int {override}	Ordena puntos descendentes y luego nombre ascendente. Si ambos coinciden devuelve 0.

Los parámetros otro de compareTo no admiten null. El empate completo en nombre y puntos puede producir 0 entre equipos diferentes; se ordena una List, no un TreeSet, para no perder equipos. Tampoco se utiliza un conjunto ordenado para almacenar eventos.

9. Métodos de encuentros y deportes

Encuentro

Firma con tipo y visibilidad	*Propósito y justificación*
+ getId(): String {override}	Implementa identidad de Programable.
+ getProgramacion(): Optional<Programacion> {override}	Delega consulta en agenda.
+ getEquiposOcupados(): Set<Equipo> {override}	Devuelve los dos equipos en un conjunto inmodificable.
+ getDescripcion(): String {override}	Usa getDeporte y nombres de ambos equipos.
+ getEstado(): EstadoEncuentro	Deriva el estado de marcador y agenda.
+ getMarcadorA(): Optional<Integer>	Vacío antes del resultado; valor del primer equipo después.
+ getMarcadorB(): Optional<Integer>	Vacío antes del resultado; valor del segundo equipo después.
+ programar(fecha: LocalDate, hora: int, sede: String): void {override, final}	Rechaza FINALIZADO y delega programación en agenda.
+ registrarResultado(a: int, b: int): void {override, final}	Plantilla común: estado, no negativos, validarDeporte, puntosPara en ambos sentidos, capacidad y confirmación.
# validarDeporte(a: int, b: int): void {abstract}	Punto variable: rechaza marcador incompatible con el deporte.
# puntosPara(favor: int, contra: int): int {abstract}	Punto variable: aporta puntos al equipo indicado; sin modificar estado.
# getDeporte(): String {abstract}	Etiqueta deportiva para una descripción polimórfica.

FutbolSala

Firma con tipo y visibilidad	*Propósito y justificación*
# validarDeporte(a: int, b: int): void {override}	No impone restricción adicional al control común de no negativos.
# puntosPara(favor: int, contra: int): int {override}	Victoria 3, empate 1 y derrota 0.
# getDeporte(): String {override}	Devuelve la etiqueta Fútbol sala.

Baloncesto

Firma con tipo y visibilidad	*Propósito y justificación*
# validarDeporte(a: int, b: int): void {override}	Rechaza marcador empatado; no altera datos.
# puntosPara(favor: int, contra: int): int {override}	Victoria 2 y derrota 1; la igualdad ya fue rechazada por validarDeporte.
# getDeporte(): String {override}	Devuelve la etiqueta Baloncesto.

10. Métodos de coordinación y presentación

Copa

Firma con tipo y visibilidad	*Propósito y justificación*
+ registrarEquipo(codigo: String, nombre: String, empresa: String): Equipo	Valida código único; construye, agrega y devuelve equipo.
+ crearFutbolSala(id: String, codA: String, codB: String): FutbolSala	Valida id, resuelve equipos y registra nuevo encuentro.
+ crearBaloncesto(id: String, codA: String, codB: String): Baloncesto	Mismo control de alta mediante helper; selecciona la subclase.
+ crearExhibicion(id: String, titulo: String): Exhibicion	Valida id; construye exhibición vinculada a esta copa y la registra.
+ buscarEvento(id: String): Programable	Devuelve evento registrado o rechaza id inexistente.
+ buscarPuntuable(id: String): Puntuable	Resuelve en el índice tipado; rechaza exhibición o id inexistente sin consultar tipos.
~ validarAgenda(evento: Programable, candidata: Programacion): void	Exige pertenencia; excluye evento por identidad; comprueba sede y participantes contra cada evento con agenda.
+ getTabla(): List<Equipo>	Copia equipos; ordena por Equipo.compareTo; devuelve lista inmodificable.
+ iterator(): Iterator<Programable> {override}	Copia solo programados; ordena por Programable.compareTo; devuelve iterador inmodificable.
- validarId(id: String): void	Centraliza id de evento no vacío y único.
- buscarEquipo(codigo: String): Equipo	Normaliza código, resuelve en mapa o rechaza inexistencia.

Cada alta deportiva registra la misma instancia en eventos y puntuables una vez validado todo. Una exhibición se registra solo en eventos. El índice de Puntuable es una vista de acceso tipada mantenida en el alta, no una segunda copia de los datos. Main usa buscarPuntuable(id).registrarResultado(a,b); así no necesita conversiones ni comprobaciones instanceof.

11. Métodos de programación y entrada

Exhibicion

Firma con tipo y visibilidad	*Propósito y justificación*
+ getId(): String {override}	Implementa identidad.
+ getProgramacion(): Optional<Programacion> {override}	Delega consulta en agenda.
+ getEquiposOcupados(): Set<Equipo> {override}	Devuelve conjunto vacío e inmodificable.
+ getDescripcion(): String {override}	Devuelve titulo.
+ getEstado(): EstadoExhibicion	SIN_PROGRAMAR si agenda vacía; PROGRAMADA si hay reserva.
+ programar(fecha: LocalDate, hora: int, sede: String): void {override}	Delega en Agenda; permite repetir la operación con datos válidos.

Agenda

Firma con tipo y visibilidad	*Propósito y justificación*
~ getActual(): Optional<Programacion>	Envuelve actual sin exponer una referencia mutable.
~ programar(evento: Programable, fecha: LocalDate, hora: int, sede: String): void	Crea candidata, pide Copa.validarAgenda y reemplaza actual solo al aprobar; evento debe estar registrado.

Programacion

Firma con tipo y visibilidad	*Propósito y justificación*
+ getFecha(): LocalDate	Devuelve valor de fecha inmutable.
+ getHora(): int	Devuelve la hora.
+ getSede(): String	Devuelve nombre de sede.

Main

Firma con tipo y visibilidad	*Propósito y justificación*
+ main(args: String[]): void {static}	Crea Copa, carga datos iniciales, mantiene menú y delega operaciones.
- cargarDatos(copa: Copa): void {static}	Prepara el conjunto inicial de la sección 12 usando altas y resultados válidos.
- ejecutarMenu(copa: Copa, entrada: Scanner): void {static}	Lee, convierte, invoca contratos y muestra consultas. Captura errores de formato y validación y continúa.

Constructor y visibilidad	*Propósito e inicialización*
+ Copa()	Inicializa los tres mapas vacíos. Los mapas de eventos y puntuables referencian el mismo objeto para cada encuentro.
~ Equipo(codigo: String, nombre: String, empresa: String)	Valida textos no vacíos, fija datos y pone puntosTabla en 0. Copa valida unicidad antes de registrar.
~ Encuentro(id: String, a: Equipo, b: Equipo, copa: Copa)	Exige id, equipos y copa válidos; a distinto de b; fija participantes, crea Agenda(copa) y deja ambos marcadores en null. No instanciable directamente.
~ FutbolSala(id: String, a: Equipo, b: Equipo, copa: Copa)	Invoca super(id, a, b, copa); no añade atributos ni duplica inicialización.
~ Baloncesto(id: String, a: Equipo, b: Equipo, copa: Copa)	Invoca super(id, a, b, copa); no añade atributos ni duplica inicialización.
~ Exhibicion(id: String, titulo: String, copa: Copa)	Valida id, título y copa, fija textos y crea Agenda(copa) vacía. No inicializa equipos ni marcador porque no los posee.
~ Agenda(copa: Copa)	Exige copa no nula y comienza sin programación. Solo la entidad

	propietaria utiliza esta agenda.
+ Programacion(fecha: LocalDate, hora: int, sede: String)	Valida todo antes de construir; objeto inmutable que reemplaza una reserva completa.
- Main()	Evita instancias; la entrada y auxiliares son estáticos.

Secuencia de creación: Main crea Copa; Copa valida ids y equipos, invoca el constructor concreto y registra el objeto solo cuando la construcción termina. FutbolSala y Baloncesto llaman al constructor de Encuentro mediante super; este inicializa equipos y marcador y crea Agenda con la misma copa. Exhibicion crea su propia Agenda sin entrar en la jerarquía deportiva.

Programable y Puntuable no tienen constructores. Las enumeraciones usan su construcción implícita; no se crean mediante new. Los constructores no llaman métodos abstractos ni registran resultados. Los estados iniciales se deducen de agenda vacía y marcador ausente.

12. Justificación y extensibilidad

Por qué se separan las dos interfaces

Programable expresa la capacidad de ocupar una sede y aparecer en el calendario. Puntuable expresa la capacidad de recibir un marcador y repartir puntos. Encuentro cumple ambas; Exhibicion solo cumple la primera. Esta separación impide imponer datos deportivos a una clínica o demostración y mantiene cada cliente dependiente de las operaciones que necesita.

Si una exhibición implementara Puntuable, tendría que inventar equipos y resultados, ignorar la operación o lanzar siempre una excepción por ser exhibición. Ninguna opción representa el caso. Con dos contratos, esa operación no existe para Exhibicion y la búsqueda en el índice de puntuables puede rechazarla antes de intentar registrar un marcador.

Extensión	***Crear o modificar***	***Elementos que permanecen iguales***
Tercer deporte con dos equipos y dos enteros	Crear una subclase de Encuentro con constructor, validarDeporte, puntosPara y getDeporte. Añadir su operación de creación en Copa y opción de menú en Main.	Equipo.compareTo, Copa.getTabla, Copa.iterator, Programable.compareTo, Agenda y algoritmo común de resultados.
Deporte con sets u otro marcador	Revisar la representación de resultado y el contrato Puntuable si dos enteros ya no bastan. Es un cambio de alcance, no solo una regla de puntos.	No se promete extensión sin cambios fuera del modelo de marcador fijado por el PDF.
Nuevo evento no competitivo	Crear una clase que implemente Programable y use Agenda; declarar sus recursos ocupados y descripción. Añadir alta en Copa y menú.	Calendario, criterio cronológico y tabla de posiciones.
Nuevo evento que reparte puntos	Además de Programable, cumplir Puntuable e incorporarlo al índice tipado al crear. Definir participantes y algoritmo correspondiente.	Los consumidores continúan trabajando contra contratos.

Herencia y composición

La clase abstracta comparte implementación exclusivamente entre encuentros. Exhibicion no hereda de Encuentro. Agenda comparte programación por composición y Programacion evita repetir los campos fecha, hora y sede en cada evento. Copa utiliza conjuntos de participantes para conflictos y compareTo para ordenar; no pregunta por el deporte ni por la clase de evento.

13. Escenario de coherencia del diseño

Conjunto propuesto para una futura carga inicial. Sirve para revisar las reglas sobre papel; no representa datos cargados ni pruebas ejecutadas.

Código	*Equipo*	*Empresa*
FAR	Faro	Empresa Norte
SOL	Sol	Empresa Sur
LUN	Luna	Empresa Norte
RIO	Río	Empresa Sur

Evento	*Programación propuesta*	*Marcador y aporte esperado*
E1 · Fútbol · Faro / Sol	12 nov 2026 · 9 · Arena A	2–2; Faro +1, Sol +1. Finalizado.
E2 · Baloncesto · Luna / Río	12 nov 2026 · 10 · Arena A	78–74; Luna +2, Río +1. Finalizado.
E3 · Fútbol · Faro / Luna	13 nov 2026 · 9 · Arena A	Sin resultado. Programado.
E4 · Baloncesto · Sol / Río	Sin programación	Sin resultado. Sin programar.
X1 · Clínica de baloncesto	12 nov 2026 · 9 · Arena B	Exhibición programada, sin equipos ni puntos.

Tabla esperada: Luna 2; Faro 1; Río 1; Sol 1. El orden de los tres últimos lo decide el nombre. Calendario esperado: E1, X1, E2, E3. E4 no aparece. E1 y X1 comparten fecha y hora, pero ocupan sedes distintas; la sede decide su orden.

Revisión conceptual	*Conclusión esperada*
Mover E3 a 12 nov 2026, 9, Arena C	Rechazo por Faro ocupado en E1; E3 conserva 13 nov, 9, Arena A.
Mover X1 a 12 nov 2026, 10, Arena A	Rechazo por sede ocupada en E2; conserva Arena B a las 9.
Mover X1 a 13 nov 2026, 11, Arena B	Válido; calendario E1, E2, E3, X1.
Dar otro resultado a E1 o resultado a E4	Rechazo por finalizado o sin programar; la tabla no cambia.
Programar E4 en un horario libre y registrar 60–60	La programación es válida; el resultado se rechaza y E4 sigue PROGRAMADO, sin marcador.

14. UML general de contratos y herencia

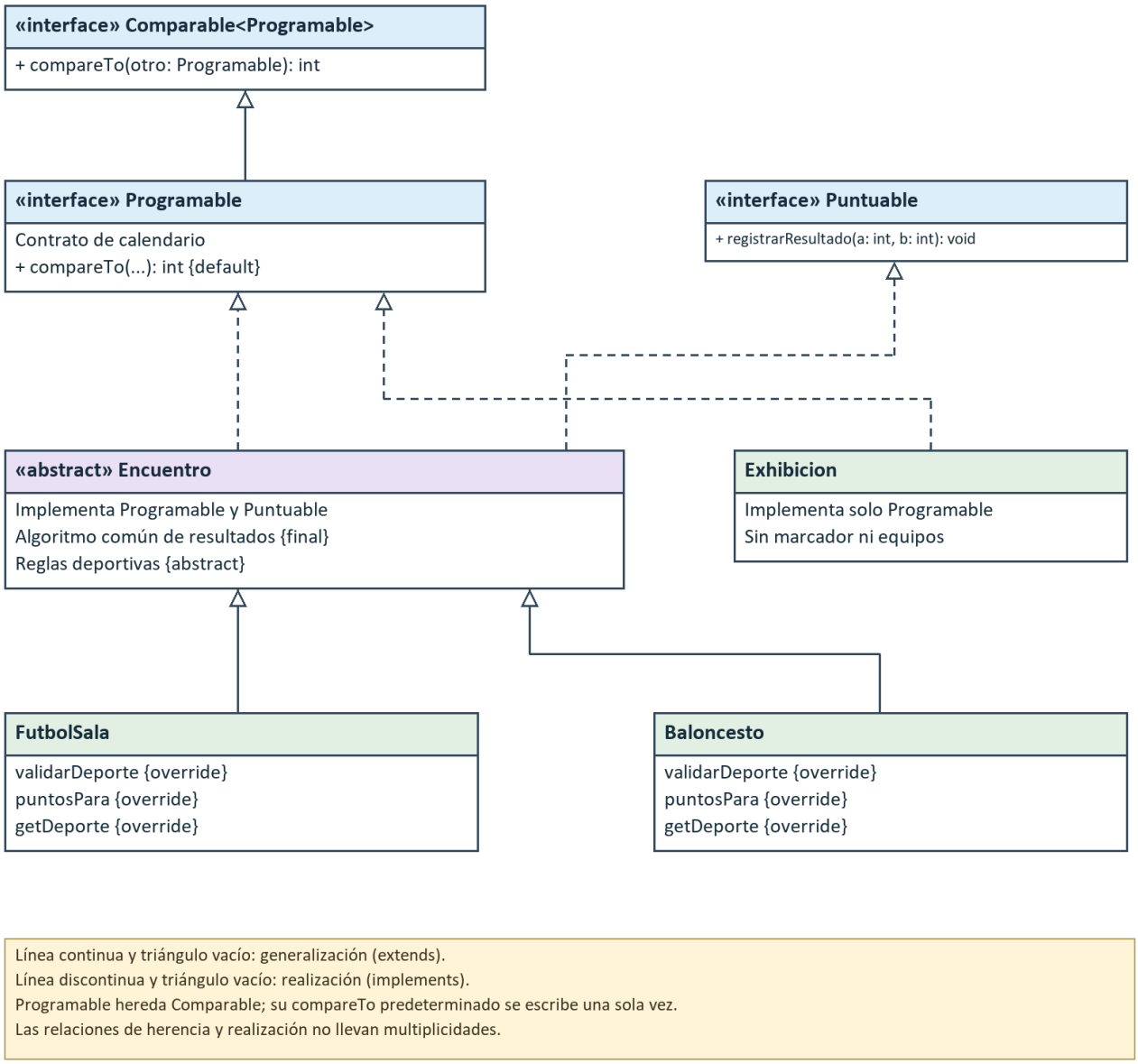


Figura 1. Contratos y jerarquía. Las vistas de detalle siguientes completan los miembros de cada tipo.

15. UML de interfaces y enumeraciones

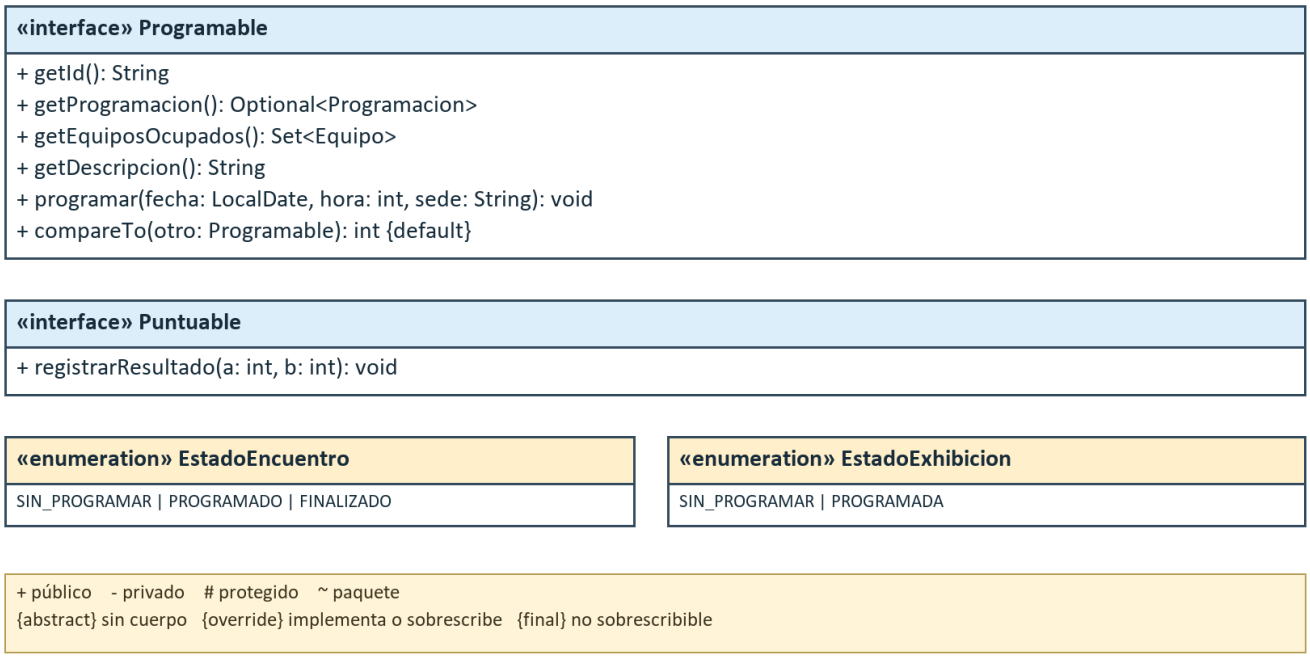


Figura 2. Miembros declarados de Programable, Puntuable. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

16. UML del encuentro abstracto

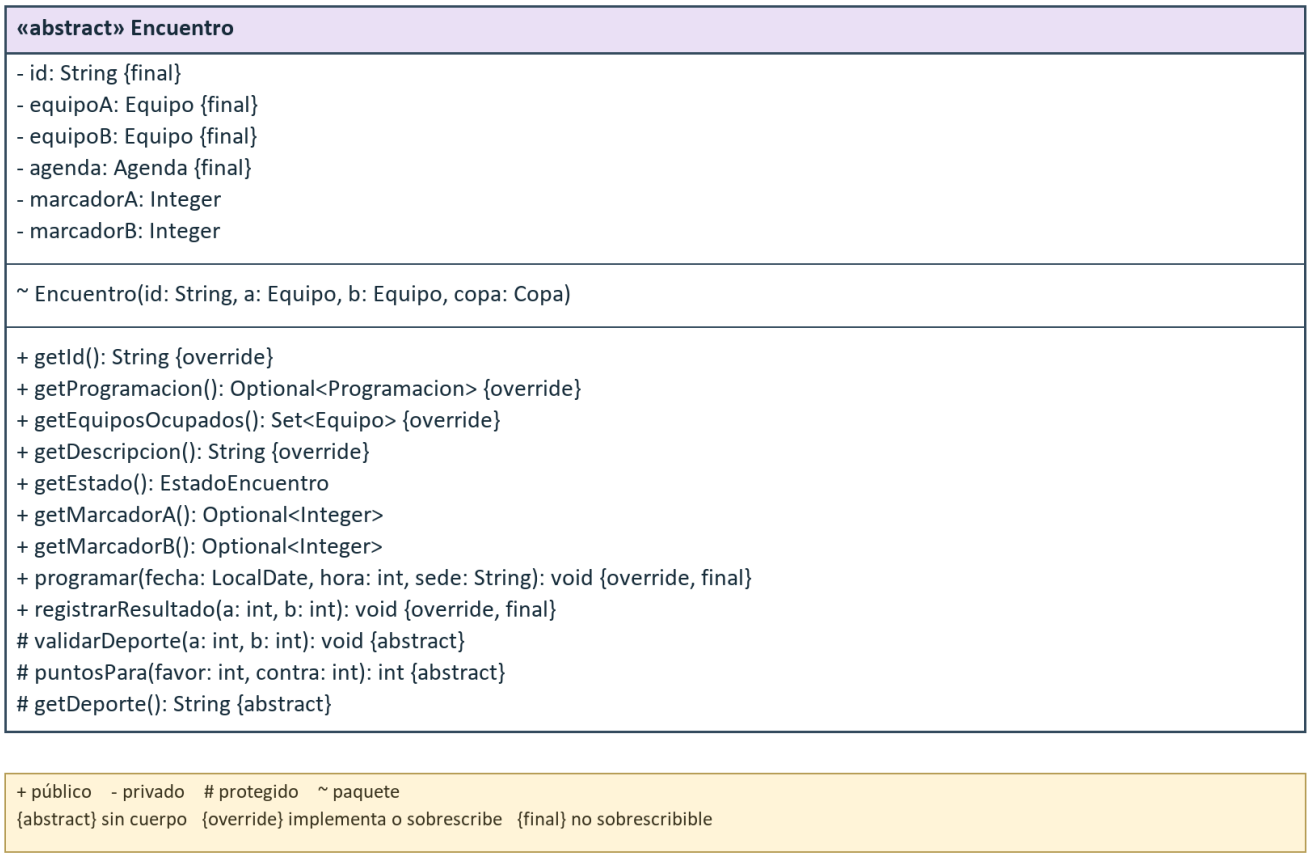


Figura 3. Miembros declarados de Encuentro. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

17. UML de deportes y exhibición

FutbolSala
~ FutbolSala(id: String, a: Equipo, b: Equipo, copa: Copa)
validarDeporte(a: int, b: int): void {override} # puntosPara(favor: int, contra: int): int {override} # getDeporte(): String {override}

Baloncesto
~ Baloncesto(id: String, a: Equipo, b: Equipo, copa: Copa)
validarDeporte(a: int, b: int): void {override} # puntosPara(favor: int, contra: int): int {override} # getDeporte(): String {override}

Exhibicion
- id: String {final} - titulo: String {final} - agenda: Agenda {final}
~ Exhibicion(id: String, titulo: String, copa: Copa)
+ getId(): String {override} + getProgramacion(): Optional<Programacion> {override} + getEquiposOcupados(): Set<Equipo> {override} + getDescripcion(): String {override} + getEstado(): EstadoExhibicion + programar(fecha: LocalDate, hora: int, sede: String): void {override}

+ público - privado # protegido ~ paquete
{abstract} sin cuerpo {override} implementa o sobrescribe {final} no sobrescribible

Figura 4. Miembros declarados de FutbolSala, Baloncesto, Exhibicion. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

18. UML de equipo y programación

Equipo
- codigo: String {final} - nombre: String {final} - empresa: String {final} - puntosTabla: int
~ Equipo(codigo: String, nombre: String, empresa: String)
+ getCodigo(): String + getNombre(): String + getEmpresa(): String + getPuntosTabla(): int ~ acumular(puntos: int): void + compareTo(otro: Equipo): int {override}

Programacion
- fecha: LocalDate {final} - hora: int {final} - sede: String {final}
+ Programacion(fecha: LocalDate, hora: int, sede: String)
+ getFecha(): LocalDate + getHora(): int + getSede(): String

+ público - privado # protegido ~ paquete
{abstract} sin cuerpo {override} implementa o sobrescribe {final} no sobrescribible

Figura 5. Miembros declarados de Equipo, Programacion. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

19. UML de copa

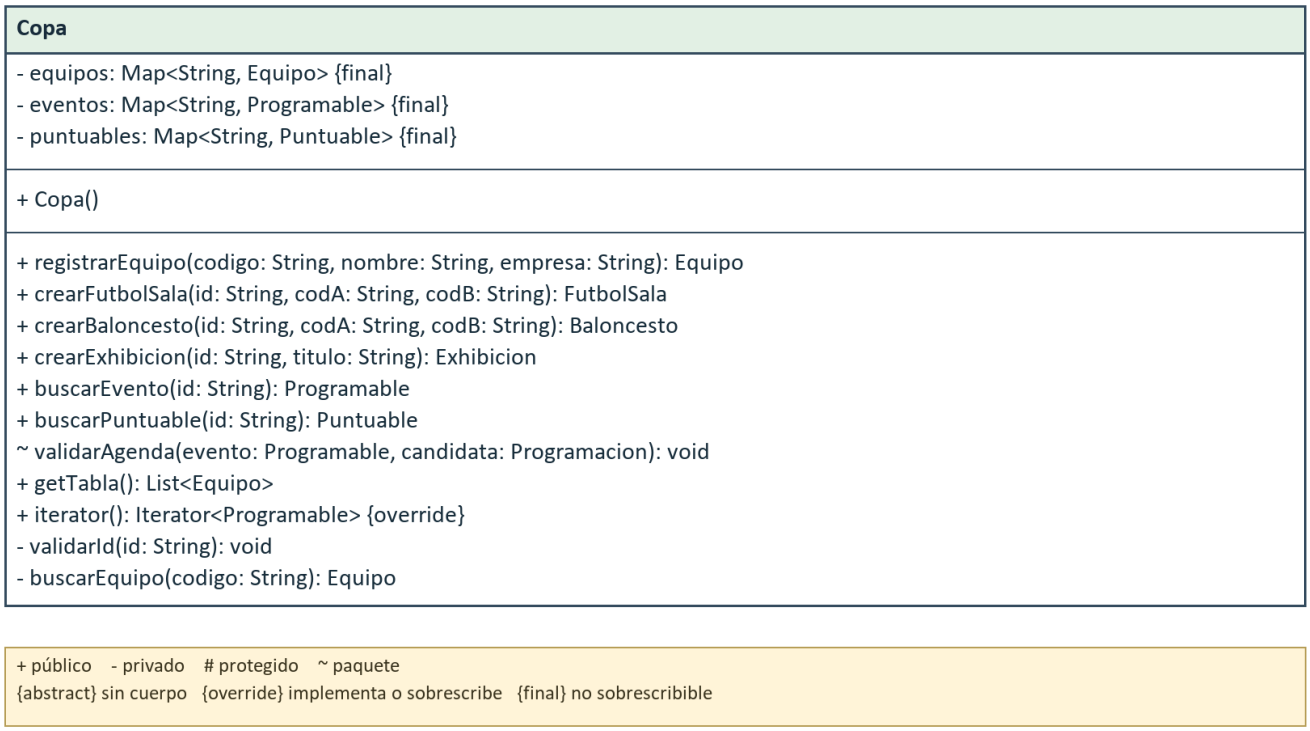


Figura 6. Miembros declarados de Copa. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

20. UML de auxiliares y entrada

Agenda
- copa: Copa {final} - actual: Programacion
~ Agenda(copa: Copa)
~ getActual(): Optional<Programacion> ~ programar(evento: Programable, fecha: LocalDate, hora: int, sede: String): void

Main
- Main()
+ main(args: String[]): void {static} - cargarDatos(copa: Copa): void {static} - ejecutarMenu(copa: Copa, entrada: Scanner): void {static}

+ público - privado # protegido ~ paquete
{abstract} sin cuerpo {override} implementa o sobrescribe {final} no sobrescribible

Figura 7. Miembros declarados de Agenda, Main. Los miembros heredados no se repiten; tipos abreviados de java.util y java.time.

21. UML de asociaciones y multiplicidades

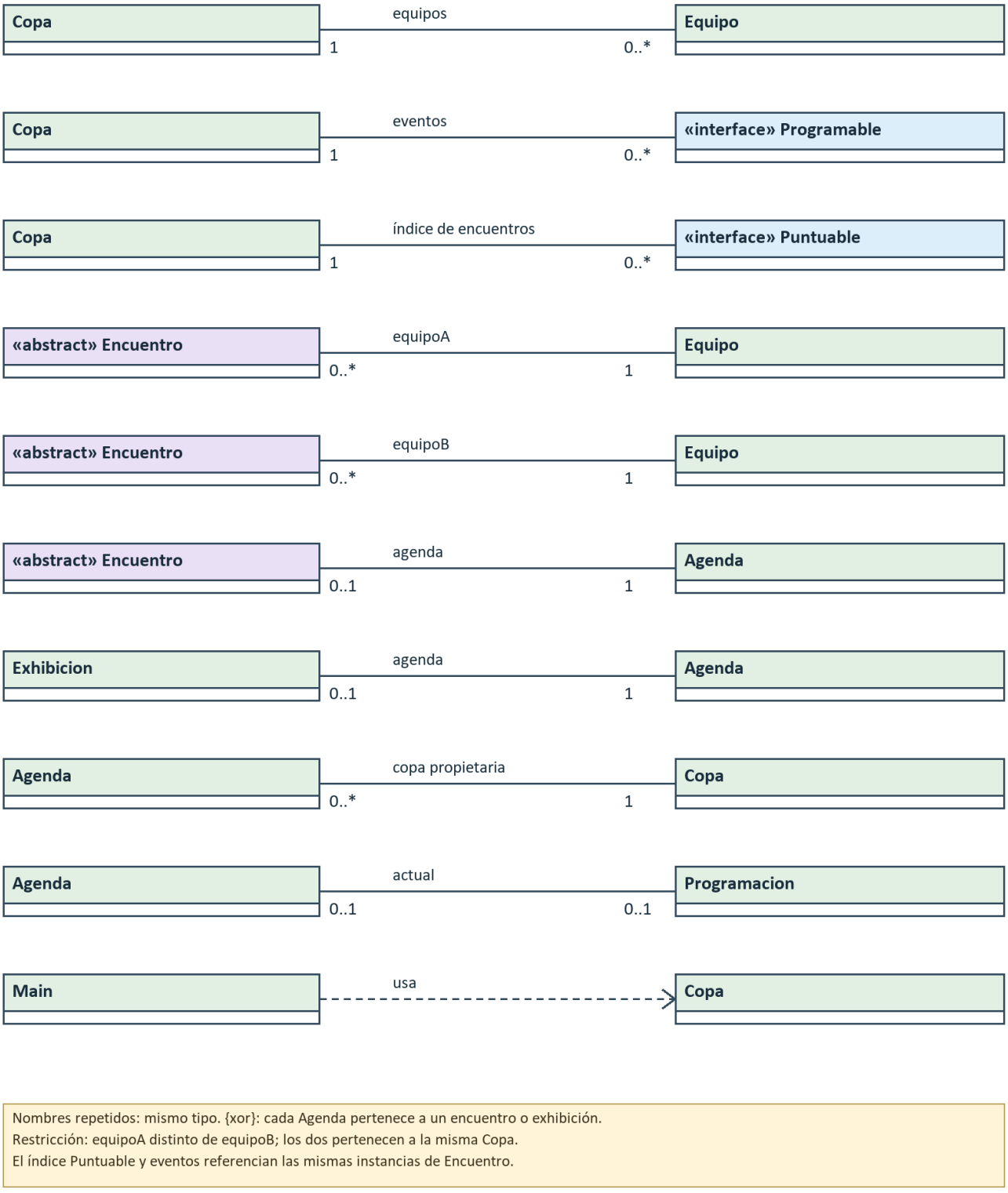
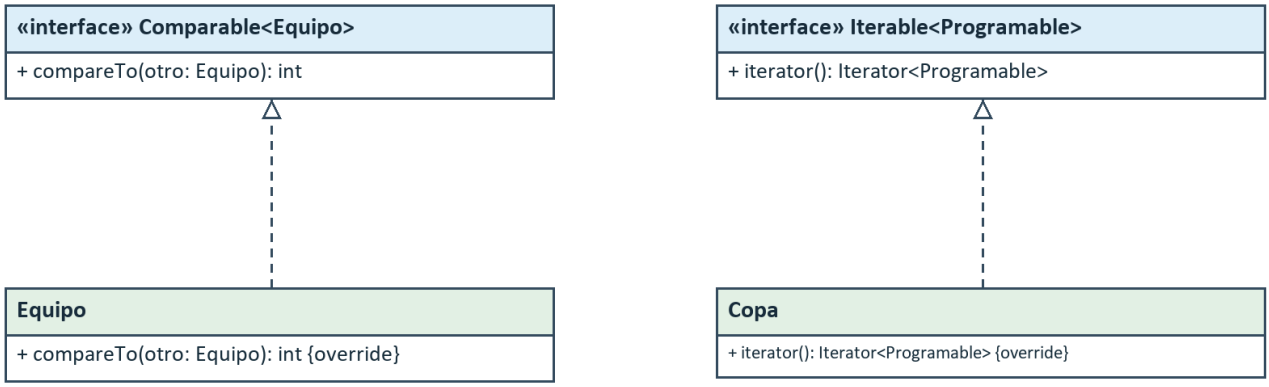


Figura 8. Asociaciones con multiplicidades en cada extremo. Main usa Copa mediante una dependencia discontinua con flecha abierta. Las agendas pertenecen a un solo evento.

22. UML de contratos de colección y anotaciones



Orden de tabla: Equipo.compareTo → puntos descendentes, nombre ascendente.
Orden de calendario: Programable.compareTo {default} → fecha, hora, sede.
Copa.iterator filtra programados y ordena una copia usando el contrato común.
No se conservan listas ordenadas mutables: cada consulta refleja los datos vigentes.

Encuentro.registrarResultado valida todo antes de confirmar el resultado.
FutbolSala.puntosPara devuelve 3 / 1 / 0; Baloncesto.puntosPara devuelve 2 / 1.
Agenda centraliza programación; Copa.validarAgenda centraliza conflictos.
El cliente usa Programable o Puntuable; no selecciona comportamiento por tipo.

Figura 9. Realización de contratos de colección y anotaciones de decisiones. Las flechas discontinuas terminan en triángulos vacíos.